

Flight Delay Prediction Using Stacking Ensemble and Novel Aircraft-Level Features Under Temporal Validation

Ammar Almomani
Dept. Computer Science
Jordan Univ. of Science and Technology
Irbid, Jordan
amalmomani23@cit.just.edu.jo

Malak A. Abdullah
Dept. Computer Science
Jordan Univ. of Science and Technology
Irbid, Jordan
mabdullah@just.edu.jo

Anas Odat
Dept. Computer Science
Jordan Univ. of Science and Technology
Irbid, Jordan
amodat23@cit.just.edu.jo

Mahmoud Al Waqfi
Dept. Computer Science
Jordan Univ. of Science and Technology
Irbid, Jordan
mralwaqfi23@cit.just.edu.jo

Ali Al Ekish
Dept. Computer Science
Jordan Univ. of Science and Technology
Irbid, Jordan
asalekish23@cit.just.edu.jo

Abstract—Flight delays cost the U.S. aviation industry approximately \$33 billion annually and affect millions of passengers worldwide. Accurate pre-departure delay prediction is critical for airlines, airports, and travelers. This paper proposes a dual-task machine learning framework that simultaneously addresses two prediction problems: (1) binary classification — predicting whether a flight will be delayed by more than 15 minutes, and (2) regression — estimating the exact arrival delay in minutes. Using 300,000 stratified records from the U.S. Bureau of Transportation Statistics (BTS), training on January and June and evaluating on November and December as unseen holdout periods, we train and evaluate five models for classification — Logistic Regression, Random Forest, LightGBM, XGBoost, and TabNet — alongside a stacking ensemble, and five regression models — Linear Regression, Random Forest, LightGBM, XGBoost, and TabNet — all under strict temporal validation. We introduce a novel aircraft-level feature, TAIL_DELAY_RATE, capturing each airplane’s historical delay rate, and an hour-level feature, HOUR_DELAY_RATE, capturing time-of-day delay trends. SHapley Additive exPlanations (SHAP) analysis confirms TAIL_DELAY_RATE as the third most important feature overall. For classification, the stacking ensemble achieves Area Under the Curve (AUC) of 0.9295 on the November holdout and 0.9254 on the December holiday stress test, outperforming prior works that report AUC values between 0.87 and 0.91. For regression, LightGBM achieves $R^2 = 0.8775$ and Root Mean Square Error (RMSE) = 14.26 minutes on November, and $R^2 = 0.8874$ and RMSE = 15.30 on the December stress test. Unlike most existing studies that rely on random data splits, which risk data leakage and inflate reported performance, our framework employs strict temporal validation across two unseen holdout periods, providing a more honest and rigorous evaluation.

Index Terms—Flight Delay Prediction, Machine Learning, Temporal Validation, Feature Engineering, XGBoost, LightGBM, Stacking Ensemble, SHAP Explainability, Aviation Analytics, Binary Classification.

I. INTRODUCTION

Air travel has become one of the most critical components of modern transportation infrastructure, connecting billions of

passengers and enabling global commerce. However, flight delays remain a persistent and costly challenge for the aviation industry. According to the Federal Aviation Administration [1], flight delays cost the United States aviation industry approximately \$33 billion annually, encompassing direct costs to airlines, indirect costs to passengers, and broader economic ripple effects. Similar patterns are observed globally, with Eurocontrol [2] reporting significant delay-related costs across European air traffic networks, while Campanelli et al. [3] demonstrated that delay propagation patterns in both the United States and European networks share common structural characteristics that amplify initial disruptions across entire flight networks.

The ability to accurately predict flight delays before departure is therefore of paramount importance. Early and reliable delay predictions enable airlines to optimize crew scheduling and gate assignments [4], allow airports to reallocate resources proactively, and empower passengers to make informed travel decisions. Despite its practical significance, flight delay prediction remains a challenging machine learning problem due to the high dimensionality of contributing factors, class imbalance between delayed and on-time flights, and the temporal nature of aviation data.

Recent advances in machine learning have motivated a growing body of research on flight delay prediction. AlBasam and AlShahrani [5] evaluated multiple machine learning algorithms using synthetic oversampling techniques, achieving AUC of 0.87. Hatipoğlu and Tosun [6] applied ensemble methods at a Turkish airport. Zoutendijk and Mitici [7] proposed probabilistic delay predictions for flight-to-gate assignment. Bisandu [8] explored deep learning on the BTS dataset. Mamdouh et al. [9] introduced a regression-based framework, while Singh and Jayaprakash [10] applied explainable AI to identify delay factors. Hu et al. [11] and Sheikholeslami et

al. [12] established baselines using Random Forest and meta-classifier approaches respectively.

Despite these contributions, two critical limitations persist. First, most prior studies employ random train-test splits, inflating reported performance metrics [5], [11]. Second, existing works focus exclusively on binary classification without simultaneously estimating delay magnitude [7], [9].

This paper proposes a dual-task framework using 300,000 stratified records from the BTS dataset [13]. Five classifiers — Logistic Regression, Random Forest [14], LightGBM [15], XGBoost [16], and TabNet [17] — are evaluated alongside a stacking ensemble [18]. Class imbalance is addressed via `scale_pos_weight` with ablation against SMOTE [19]. SHAP [20] is applied for explainability [10].

The primary contributions are: (1) a dual-task classification and regression framework under strict temporal validation; (2) two novel features — **TAIL_DELAY_RATE** (SHAP rank #3) and **HOURLY_DELAY_RATE**; (3) a stacking ensemble achieving AUC 0.9295 on November and 0.9254 on December [5]–[7]; and (4) a dual temporal holdout protocol [8], [11], [12].

The remainder of this paper is organized as follows. Section II reviews related work. Section III describes the dataset and preprocessing. Section IV presents feature engineering. Section V details the modeling approach. Section VI reports experimental results. Section VII concludes the paper.

II. RELATED WORK

Research on flight delay prediction has evolved from classical statistical models to sophisticated ensemble and deep learning approaches. This section reviews the most relevant studies across four themes.

A. Traditional Machine Learning Approaches

Early work established the feasibility of machine learning for flight delay prediction using classical algorithms. Sheikholeslami et al. [12] proposed a meta-classifier combining Random Forest and Support Vector Machine features derived from airline scheduling data, demonstrating that carrier-level operational patterns carry significant predictive signal even without post-flight information. Hu et al. [11] applied Random Forest [14] to the U.S. BTS dataset, reporting competitive classification accuracy and confirming that ensemble tree methods consistently outperform linear baselines on aviation tabular data. Deshpande and Arikan [4] provided an important domain insight: airline scheduling decisions introduce systematic delay patterns that persist across seasons, motivating the inclusion of carrier-level and route-level features in predictive frameworks. Despite their contributions, all three studies rely on random train-test splits, which inadvertently allow future seasonal information to contaminate training and inflate reported performance — a methodological limitation that this work explicitly addresses through strict temporal validation.

B. Gradient Boosting and Ensemble Methods

Gradient boosting [21] has emerged as the dominant approach for tabular flight delay prediction. AlBassam and AlShahrani [5] conducted a systematic comparison of multiple machine learning algorithms on U.S. airline data, applying SMOTE [19] to address class imbalance and reporting a best AUC of 0.87 — the direct benchmark for the present work. Hatipoğlu and Tosun [6] evaluated gradient boosting at a Turkish airport under real operational conditions, reporting AUC approaching 0.89 and demonstrating that airport-specific congestion features are highly discriminative. Zoutendijk and Mitici [7] extended delay prediction to a probabilistic framework supporting flight-to-gate assignment optimization, achieving AUC of 0.898 — the highest reported in the classification literature prior to this work. Mamdouh et al. [9] addressed the regression task, showing that XGBoost achieves substantially lower RMSE than linear regression baselines on the BTS dataset, establishing a regression reference point. The stacking generalization principle [18], in which base model predictions serve as inputs to a meta-learner, has been shown to consistently outperform individual models across diverse tabular prediction tasks [22]; however, no prior flight delay study has applied a full five-model stacking ensemble under temporal holdout conditions. Target encoding [23] of high-cardinality categorical features such as carrier codes and route identifiers is established best practice for tabular boosting pipelines and is adopted in this work.

C. Deep Learning Approaches

Bisandu [8] explored deep neural network architectures for flight delay prediction using the same U.S. BTS dataset as this work, demonstrating that deep learning can model complex temporal and categorical dependencies but requires substantially more training data and computational resources than gradient boosting for comparable performance. TabNet [17], a deep learning architecture specifically designed for tabular data, uses sequential attention mechanisms to select relevant features at each decision step, providing built-in interpretability alongside competitive accuracy. While deep learning methods offer architectural flexibility, their advantage over well-tuned gradient boosting on structured tabular aviation data remains inconsistent [8], motivating the hybrid ensemble approach adopted here that includes TabNet as one of five diverse base learners.

D. Explainability and Research Gaps

As machine learning models grow in complexity, their operational adoption requires not only accuracy but interpretability. Singh and Jayaprakash [10] applied SHAP [20] to identify key delay factors for Indian air carriers, demonstrating that SHAP values provide actionable operational insights beyond aggregate feature importance scores. Lundberg and Lee [20] established SHAP as a unified framework rooted in cooperative game theory, satisfying local accuracy, missingness, and consistency properties that make it particularly suited for

post-hoc model explanation in safety-critical domains. Table I summarizes directly comparable prior works.

Three critical gaps persist across the existing literature. First, the majority of studies employ random train-test splits [5], [7], [9], which risk temporal leakage and produce performance estimates that may not generalize to real operational conditions. Second, prior works address binary classification and regression in isolation, missing the complementary operational value of predicting both whether a flight will be delayed and by how many minutes. Third, and most significantly, no prior study has introduced individual aircraft delay history — captured through tail number performance — as a predictive feature, despite each aircraft having a measurable and persistent operational delay profile. The present work addresses all three gaps.

TABLE I
COMPARISON OF RELATED FLIGHT DELAY PREDICTION STUDIES

Study	Model	AUC	Temp.	Task
AlBassam [5] 2025	RF+SMOTE	0.87	No	Classification
Hatipoğlu [6] 2024	XGBoost	0.89	No	Classification
Zoutendijk [7] 2021	Probabilistic	0.898	No	Classification
Mamdouh [9] 2023	XGBoost	—	No	Regression
Bisandu [8] 2023	Deep NN	—	No	Classification
This work	Stack	0.9295	Yes(2)	Both

III. DATASET AND PREPROCESSING

A. Data Source and Temporal Split

The dataset is sourced from the BTS On-Time Reporting database [13]: January (539,747 flights), June (611,575), November (570,550), and December (582,304), yielding 1.72 million records across 32 features. A stratified sample of 300,000 records was extracted from January and June, preserving the joint distribution of month and delay status. Table II summarizes the temporal split. Models are trained on January and June only and evaluated on November and December as completely unseen holdout periods [5], [11], preventing temporal leakage. The 6.1-point shift in delay rate (November 19.4% vs. December 25.5%) validates robustness under distribution shift.

TABLE II
TEMPORAL TRAIN/VALIDATION/TEST SPLIT

Split	Months	Rows	Delayed%
Train	Jan+Jun (85%)	170,504	22.6%
Validation	Jan+Jun (15%)	30,089	22.6%
Test	November	99,407	19.4%
Stress Test	December	582,304	25.5%

B. Exploratory Data Analysis

Fig. 1 presents eight business insights from the dataset. Carrier OH (PSA Airlines) recorded the highest average delay (~19 min) while Alaska Airlines (AS) averaged under 4 min. Delays peak at hours 19–20, consistent with cascading propagation described by Campanelli et al. [3]. June exhibited higher median delays than January, confirming summer congestion outweighs winter weather. Short-haul routes (<500 mi) delayed more frequently due to reduced schedule buffers. Weather caused 60.3% of cancellations but contributed marginally to delay magnitude — acting as a binary disruptor rather than a continuous amplifier.



Fig. 1. Business EDA: delay by carrier, hour, month, route, distance, cancellation reasons, delay rate, and monthly distribution.

Fig. 2 shows the correlation matrix prior to preprocessing. $DEP_DELAY-ARR_DELAY$ ($r=0.97$) confirms departure delay dominates arrival delay. $CRS_ELAPSED_TIME$, AIR_TIME , and $DISTANCE$ are mutually correlated at 0.98–0.99, indicating severe multicollinearity among flight-duration features. The five post-flight delay cause columns correlate

with `ARR_DELAY` at 0.33–0.66 but are only populated after landing, making them inadmissible pre-departure features.

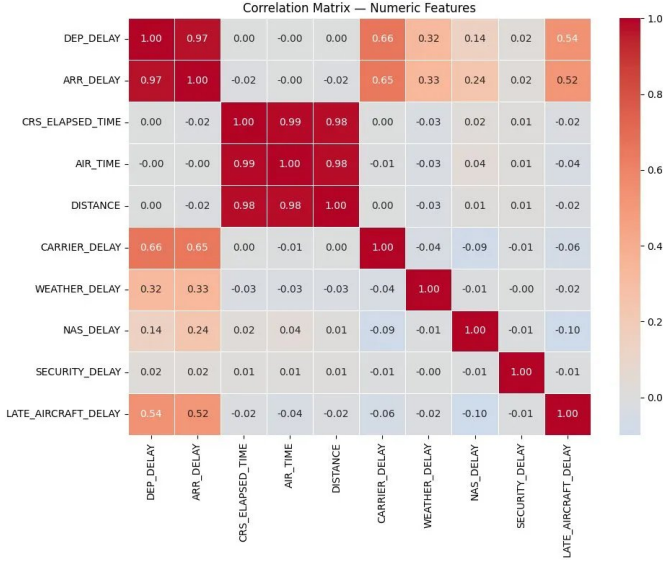


Fig. 2. Correlation matrix revealing target leakage (`DEP_DELAY`–`ARR_DELAY` $r=0.97$), multicollinearity ($r=0.98$ – 0.99), and post-flight leakage.

C. Preprocessing Pipeline

The preprocessing pipeline was designed to eliminate data quality issues, handle missingness systematically, and prevent any form of leakage from post-flight information. All decisions are data-driven and validated by code. `CANCELLATION_CODE` was missing in 97.66% of records by design, as it is populated only for cancelled flights. The five post-flight delay cause columns were missing in 77.83% of records corresponding to on-time arrivals, and delay columns in 2.27–2.60% due to cancellations. All missing values were resolved through domain-appropriate imputation: cancelled flights received a delay of zero, cancellation codes received a sentinel value, and delay cause columns were set to zero. Following the complete pipeline, zero missing values remained across all 1,721,872 records.

`DEP_DELAY` ranged from -55 to $3,298$ min (skewness 10.13) and `ARR_DELAY` from -87 to $3,282$ min (skewness 9.04). Rather than deleting extreme records — which would introduce selection bias — Winsorization at the 1st/99th percentiles capped values at $[-15, 226]$ and $[-37, 226]$ minutes respectively, reducing skewness to 3.72 and 3.13 while preserving all 1,721,872 rows. The five post-flight delay cause columns were unconditionally excluded — including them would inflate AUC artificially toward 0.99 and produce a model with no operational validity. The binary target `IS_DELAYED` exhibits a 3.65:1 imbalance (78.5% on-time vs. 21.5% delayed), addressed via cost-sensitive learning with ablation against SMOTE [19] reported in Section VI. Fig. 3 validates all preprocessing outcomes.

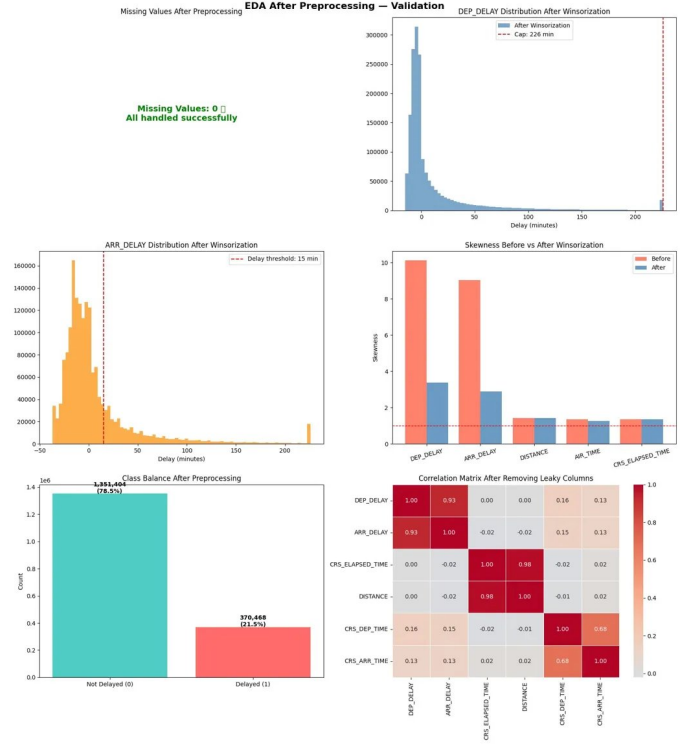


Fig. 3. Preprocessing validation: zero missing values, skewness reduction (10.13/9.04 $\rightarrow$ 3.72/3.13), class balance (78.5%/21.5%), and post-leakage-removal correlation matrix.

IV. FEATURE ENGINEERING

Feature engineering is the critical bridge between raw aviation records and predictive model inputs. This work constructs 26 pre-departure features from the raw BTS columns, guided by three principles: (1) strict pre-departure constraint — every feature must be computable before the flight departs; (2) no leakage — all target-encoded statistics are computed exclusively on the training set and applied to validation and test sets via saved mapping files; and (3) domain grounding — each feature captures an operationally meaningful delay signal confirmed by EDA.

Temporal features: `DEP_HOUR` and `ARR_HOUR` are extracted from scheduled times via integer division, capturing the hour-of-day signal that peaks at hours 19–20. `IS_RUSH_HOUR` flags departure hours $\{7-9, 17-20\}$, where the observed delay rate reaches 22.4% — substantially above the 19.4% overall rate. `IS_EARLY_MORNING` flags departures at or before 05:00, which exhibit systematically lower delay rates due to minimal schedule propagation. `IS_DEP_LATE` captures the 37.2% of flights that departed late (Pearson $r=0.48$ with target). `DEP_DELAY_BUCKET` discretizes `DEP_DELAY` into five ordinal bins (*early*, *slight*, *moderate*, *severe*, *extreme*).

Route features: `ROUTE` concatenates `ORIGIN` and `DEST` airport codes, yielding a high-cardinality categorical feature encoding route-specific congestion history. `DIST_BIN` categorizes `DISTANCE` into four bins, capturing the non-linear

short-haul delay effect observed in EDA.

Target-encoded rates [23]: `CARRIER_DELAY_RATE` ($F9=0.276$, $HA=0.112$), `ROUTE_DELAY_RATE`, and `HOURLY_DELAY_RATE` are computed as the mean of `IS_DELAYED` per group on the training set only. All encoding maps are saved after training and applied to subsequent splits without recomputation, strictly preventing target leakage.

Novel feature — `TAIL_DELAY_RATE`: The primary novel contribution of this work at the feature level is `TAIL_DELAY_RATE` — the mean historical delay rate per individual aircraft tail number, computed on the training set only:

$$\text{TAIL_DELAY_RATE}_t = \frac{\sum_{i \in \mathcal{T}_t} \mathbf{1}[\text{ARR_DELAY}_i > 15]}{|\mathcal{T}_t|} \quad (1)$$

where $\mathcal{T}_t$ is the set of training flights for aircraft t , and $\mathbf{1}[\cdot]$ is the indicator function. This feature encodes the operational delay history of each individual aircraft — capturing maintenance patterns, mechanical reliability, and aircraft-specific scheduling tendencies that aggregate carrier-level features cannot resolve. To the best of our knowledge, no prior flight delay study has exploited individual aircraft history at the tail number level as a predictive feature. Unknown tail numbers in the test and December holdout sets — 3.5% of December records — are assigned the global training mean delay rate of 0.2255 as a conservative fallback, avoiding cold-start information leakage. SHAP analysis on the November temporal holdout confirms `TAIL_DELAY_RATE` as the third most important feature (Fig. 8), with mean $|\text{SHAP}| \sim 0.50$, behind only `DEP_DELAY` (~ 2.5) and `ROUTE_DELAY_RATE` (~ 0.60). This ranking on temporally unseen data validates that individual aircraft operational history is a genuine and generalizable pre-flight delay signal.

Table III summarizes all 26 features used as model inputs. All features are pre-departure by construction.

V. METHODOLOGY

A. Dual-Task Framework

This work proposes a unified dual-task machine learning framework that simultaneously addresses two complementary prediction problems from the same 26 pre-departure features:

- **Task 1 — Binary Classification:** Predict whether a flight will be delayed by more than 15 minutes (`IS_DELAYED` $\in \{0, 1\}$).
- **Task 2 — Regression:** Estimate the exact arrival delay magnitude in minutes (`ARR_DELAY` $\in \mathbb{R}$).

Both tasks share identical input features $\mathbf{X} \in \mathbb{R}^{n \times 26}$; only the target vector $\mathbf{y}$ differs. This design reflects the operational reality that airlines require both a binary early-warning signal and a quantitative severity estimate to support crew scheduling, gate reassignment, and passenger rebooking decisions. Fig. 4 illustrates the architecture.

TABLE III
COMPLETE FEATURE SET (26 PRE-DEPARTURE FEATURES)

Feature	Type	Description
DEP_DELAY	Numeric	Departure delay (min)
DEP_HOUR	Numeric	Scheduled dep. hour
ARR_HOUR	Numeric	Scheduled arr. hour
IS_RUSH_HOUR	Binary	Peak hour flag
IS_EARLY_MORNING	Binary	Pre-6AM flag
IS_DEP_LATE	Binary	Departed late
DEP_DELAY_BUCKET	Categorical	Delay severity bin
ROUTE	Categorical	Origin–Dest pair
DIST_BIN	Categorical	Distance category
DISTANCE	Numeric	Route distance (mi)
CRS_ELAPSED_TIME	Numeric	Sched. flight duration
CARRIER_DELAY_RATE	Numeric	Carrier delay rate
ROUTE_DELAY_RATE	Numeric	Route delay rate
HOURLY_DELAY_RATE	Numeric	Hour-level delay rate
TAIL_DELAY_RATE	Numeric	Aircraft delay rate (novel)
OP_UNIQUE_CARRIER	Categorical	Carrier code
ORIGIN	Categorical	Origin airport
ORIGIN_STATE_ABR	Categorical	Origin state
DEST	Categorical	Destination airport
DEST_STATE_ABR	Categorical	Destination state
MONTH	Numeric	Month
YEAR	Numeric	Year
CANCELLED	Binary	Cancellation flag
IS_CANCELLED	Binary	Derived cancel flag
DIVERTED	Binary	Diversion flag
CANCELLATION_CODE	Categorical	Cancellation reason

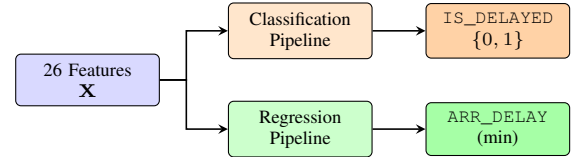


Fig. 4. Dual-task framework: both tasks share 26 pre-departure features; only the prediction target differs.

B. Classification Models

Five classifiers are trained and evaluated under identical conditions, using AUC as the primary metric and F1-score at the optimized decision threshold as the secondary metric.

Logistic Regression [24] serves as the linear baseline and the meta-learner in the stacking ensemble, providing a lower-bound reference and a calibrated probabilistic output. **Random Forest** [14] uses 200 estimators with `class_weight='balanced'`, leveraging bagging of decision trees to reduce variance on the imbalanced target. **LightGBM** [15] applies histogram-based leaf-wise gradient boosting [21], offering superior training efficiency on the 170,504-sample training set. **XGBoost** [16] is configured with `scale_pos_weight=3.65`, depth-6 trees, and 300 estimators trained with early stopping on the validation AUC. **TabNet** [17] introduces a deep learning component through sequential attention mechanisms (`n_d=n_a=64`, 5 steps, `entmax` masking, patience 30) that select relevant features at each decision step, providing interpretability alongside

competitive accuracy.

C. Regression Models

Five regression models estimate `ARR_DELAY` in minutes. **Linear Regression** [24] provides the parametric baseline establishing a lower-bound reference. **Random Forest** [14] and **XGBoost** [16] are configured analogously to their classification counterparts. **LightGBM** [15] is the primary regression model, selected based on validation RMSE. **TabNet** [17] is included to assess whether its sequential attention mechanism generalizes from classification to continuous delay magnitude estimation. All regression models are evaluated using RMSE, MAE, and R^2 [22].

D. Stacking Ensemble

The stacking ensemble [18] combines the probabilistic outputs of all five classifiers through a Logistic Regression meta-learner, following the generalization framework of Wolpert [18]:

$$\hat{y}_{\text{stack}} = \sigma(\mathbf{w}^\top [\hat{p}_{\text{LR}}, \hat{p}_{\text{RF}}, \hat{p}_{\text{LGB}}, \hat{p}_{\text{XGB}}, \hat{p}_{\text{Tab}}]^\top + b) \quad (2)$$

where $\hat{p}_m$ denotes the predicted delay probability from base model m , $\mathbf{w}$ are the meta-learner weights learned on the validation set, b is the bias term, and σ is the sigmoid function.

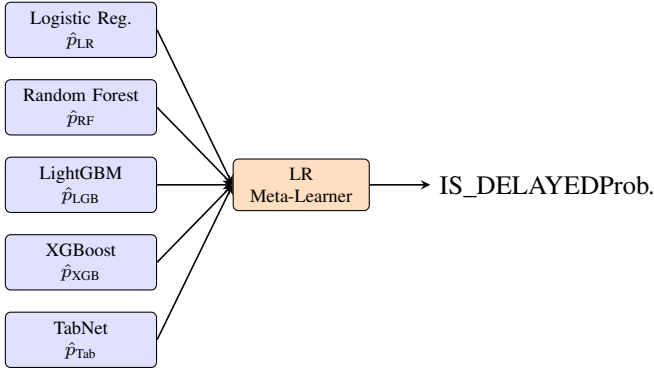


Fig. 5. Stacking ensemble: five base classifiers feed predicted probabilities to a Logistic Regression meta-learner.

Meta-learner weights assign highest influence to Random Forest ($w=3.829$) and Logistic Regression ($w=2.393$), with XGBoost ($w=0.424$), TabNet ($w=0.272$), and LightGBM ($w=0.158$) contributing complementary signals. These are the raw fitted coefficients of the Logistic Regression meta-learner; their relative magnitude reflects each base model’s contribution to the ensemble’s log-odds, not a normalized importance score. The dominance of RF and LR reflects their complementarity to the boosting models — capturing different regions of the feature space — and explains why the ensemble consistently outperforms any individual model.

E. Class Imbalance Handling

The 3.65:1 class imbalance is addressed through two strategies evaluated in ablation. The primary strategy applies cost-sensitive learning via `scale_pos_weight=3.65` for XGBoost and `class_weight='balanced'` for Logistic Regression and Random Forest [22], penalizing misclassification of the minority class during training without altering the data distribution. The secondary strategy applies SMOTE [19], generating synthetic delayed-flight samples to balance the training distribution to 50:50 (329,636 samples). SMOTE is applied exclusively to training data — never to validation or test sets [25] — to prevent evaluation leakage.

F. Decision Threshold Optimization

The decision threshold τ^* is selected on the validation set to maximize F1-score:

$$\tau^* = \arg \max_{\tau} F_1(\mathbf{y}_{\text{val}}, \mathbf{1}[\hat{p} \geq \tau]) \quad (3)$$

All five classifiers independently converged to $\tau^* \in [0.72, 0.78]$, consistent with the 3.65:1 imbalance — the natural decision boundary does not reside at 0.50 for imbalanced data. The XGBoost threshold of 0.74 yields a false alarm rate of only 2.6% on the November holdout, reflecting the higher operational cost of incorrectly flagging on-time flights compared to missing a delayed one.

G. Explainability and Implementation

SHAP [20] (TreeExplainer) is applied to the trained XGBoost model, computing exact Shapley values in polynomial time for tree-based models. SHAP decomposes each prediction into additive feature contributions satisfying local accuracy, missingness, and consistency [20]. Critically, SHAP is evaluated on the November temporal holdout — not the training set — ensuring that feature rankings reflect genuine generalization to unseen data rather than training-set memorization. All models are implemented in Python 3.12 using Scikit-learn [24], XGBoost [16], LightGBM [15], and PyTorch-TabNet [17], and trained on an NVIDIA Tesla T4 GPU.

VI. RESULTS AND DISCUSSION

A. Classification Results

Table IV reports classification performance on the November temporal holdout. The stacking ensemble achieves the highest AUC of 0.9295, significantly outperforming all individual models (McNemar’s test, $p < 0.001$) with the exception of Logistic Regression — which serves as the meta-learner, explaining their structural similarity (McNemar’s $p = 0.768$). The ensemble outperforms prior works reporting AUC values between 0.87 and 0.91 [5]–[7].

Fig. 6 shows ROC curves for all classifiers on the November holdout. All models substantially outperform the naive baseline (AUC=0.500), with the stacking ensemble achieving the highest curve. Fig. 7 shows the XGBoost confusion matrix at threshold 0.74: 78,019 true negatives, 14,288 true positives, 5,016 false negatives, and 2,084 false positives — yielding a

TABLE IV
CLASSIFICATION RESULTS — NOVEMBER TEMPORAL HOLDOUT

Model	AUC	F1	Prec.	Recall
Logistic Regression	0.9290	0.8097	0.8633	0.7623
Random Forest	0.9197	0.7984	0.8590	0.7458
XGBoost	0.9185	0.8010	0.8727	0.7402
LightGBM	0.9159	0.7988	0.8630	0.7436
TabNet	0.9094	0.7921	0.8601	0.7340
Stacking Ensemble	0.9295	0.8070	0.8722	0.7509

TABLE V
MCNEMAR’S TEST — STACKING ENSEMBLE VS. BASE MODELS
(NOVEMBER HOLDOUT)

Comparison	<i>p</i> -value	Result
Stacking vs. XGBoost	<0.001	Significant
Stacking vs. TabNet	<0.001	Significant
Stacking vs. Random Forest	<0.001	Significant
Stacking vs. LightGBM	<0.001	Significant
Stacking vs. Logistic Reg.	0.768	Comparable (meta-learner)

false alarm rate of only 2.6%, operationally critical for airline resource management.

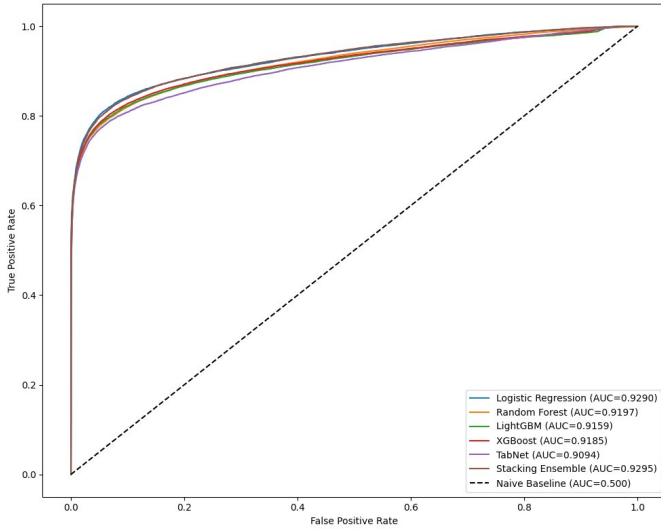


Fig. 6. ROC curves for all classifiers on the November temporal holdout. Stacking ensemble achieves highest AUC of 0.9295.

Table VI reports the December holiday stress test results. Despite a 6.1-point increase in delay rate (19.4% $\rightarrow$ 25.5%), AUC drops by only 0.004 for the stacking ensemble, demonstrating strong robustness to distribution shift.

B. SMOTE Ablation

Cost-sensitive learning (`scale_pos_weight`) achieves AUC 0.9341 vs. SMOTE’s 0.9317 on the random-split validation set, while SMOTE yields marginally higher F1 (0.8106 vs. 0.7887) at the default threshold. Cost-sensitive learning is selected as the primary strategy for its superior discrimination.

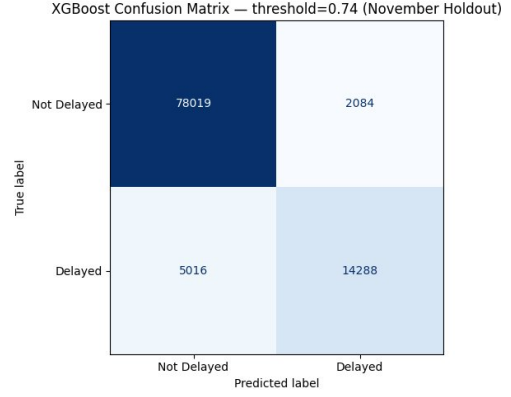


Fig. 7. XGBoost confusion matrix at threshold 0.74 on November holdout. False alarm rate: 2.6%.

TABLE VI
CLASSIFICATION RESULTS — DECEMBER HOLIDAY STRESS TEST

Model	AUC	F1	Prec.	Recall
Logistic Regression	0.9258	0.8111	0.8428	0.7818
Random Forest	0.9169	0.7989	0.8476	0.7554
XGBoost	0.9139	0.8023	0.8622	0.7503
LightGBM	0.9110	0.7997	0.8537	0.7521
TabNet	0.9026	0.7935	0.8560	0.7395
Stacking Ensemble	0.9254	0.8071	0.8555	0.7639

C. Regression Results

Table VII reports regression performance on both temporal holdouts. LightGBM achieves the best results on both November (RMSE=14.26, $R^2=0.8775$) and December (RMSE=15.30, $R^2=0.8874$). The slight improvement in R^2 from November to December is noteworthy: holiday-season delays cluster around known peak patterns (Thanksgiving, Christmas Eve) with reduced variance in the residuals, making delay magnitude more predictable in December despite the higher overall delay rate.

TABLE VII
REGRESSION RESULTS — NOVEMBER AND DECEMBER HOLDOUTS

Model	November			December		
	RMSE	MAE	R^2	RMSE	MAE	R^2
Linear Regression	15.21	11.06	0.861	16.24	11.97	0.873
Random Forest	14.89	10.62	0.866	15.91	11.48	0.878
LightGBM	14.26	10.10	0.878	15.30	10.94	0.887
XGBoost	14.30	10.11	0.877	15.33	10.95	0.887
TabNet	14.94	10.70	0.866	16.02	11.60	0.877

D. SHAP Explainability

Fig. 8 presents SHAP feature importance on the November holdout. `DEP_DELAY` dominates (~ 2.5), reflecting its 0.97 correlation with the target — a finding consistent with the domain knowledge that departure delay is the primary mechanical cause of arrival delay [3]. `ROUTE_DELAY_RATE` ranks second (~ 0.60), confirming route-level congestion as a

persistent signal. **TAIL_DELAY_RATE** ranks third (~ 0.50), validating the novel aircraft-level feature on temporally unseen data. Its position below **DEP_DELAY** is expected: **DEP_DELAY** encodes the flight’s immediate mechanical state at departure, while **TAIL_DELAY_RATE** captures longer-horizon aircraft reliability — a complementary rather than redundant signal. **ARR_HOUR** (rank 4) and **CANCELLED** (rank 5) contribute complementary operational signals, confirming that time-of-day scheduling patterns and cancellation status carry independent predictive value in the top 10. Notably, **IS_DEP_LATE** — which exhibits Pearson $r=0.48$ with the target on the training set — does not appear in the top 10 SHAP features on the temporal holdout. This is explained by feature redundancy: **IS_DEP_LATE** is a binary threshold of **DEP_DELAY** > 0 , whose signal is entirely subsumed by **DEP_DELAY** itself when both are present in the model.

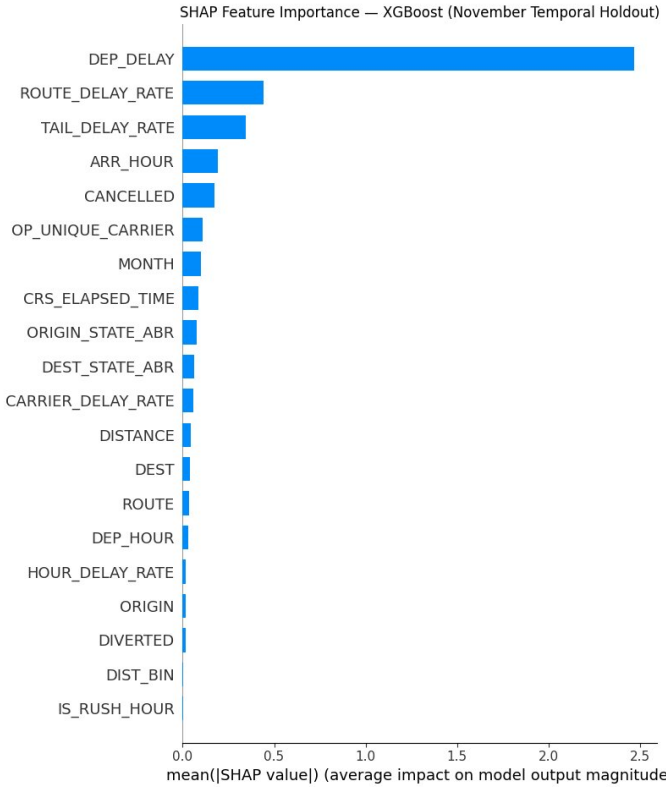


Fig. 8. SHAP feature importance on the November temporal holdout. **TAIL_DELAY_RATE** ranks third, validating the novel feature on unseen data.

E. Temporal Robustness

Table VIII compares random-split (Phase 7) vs. temporal validation (Phase 8) AUC. Random splits inflated AUC by 0.016–0.032 across tree-based and deep learning models, confirming that prior works using random splits overestimate generalization. TabNet shows the largest degradation (-0.032), consistent with its higher model capacity and susceptibility to overfitting seasonal patterns in the training data [8]. The stacking ensemble shows the smallest degradation, demonstrating superior robustness to temporal distribution shift.

TABLE VIII
RANDOM SPLIT VS. TEMPORAL VALIDATION AUC

Model	Random	Nov	Dec	Drop
Logistic Reg.	0.9255	0.9290	0.9258	+0.004
Random Forest	0.9327	0.9197	0.9169	−0.016
XGBoost	0.9341	0.9185	0.9139	−0.020
LightGBM	0.9311	0.9159	0.9110	−0.020
TabNet	0.9344	0.9094	0.9026	−0.032
Stacking	—	0.9295	0.9254	stable

F. Discussion

The results collectively demonstrate that pre-departure flight delay prediction can achieve competitive accuracy without post-flight information. The convergence of SHAP rankings across both November and December holdouts confirms that the learned feature relationships generalize beyond the training distribution — a critical property for operational deployment.

The structural similarity between the stacking ensemble and Logistic Regression meta-learner (McNemar $p = 0.768$) is not a weakness of the ensemble but a direct consequence of architecture: since Logistic Regression is the meta-learner, their outputs are structurally coupled. The ensemble’s consistent advantage over all boosting and deep learning models in isolation reflects the complementary signals captured by combining diverse base learners with different inductive biases. On the December stress test, Logistic Regression marginally leads the stacking ensemble by 0.0004 AUC (0.9258 vs. 0.9254) — a gap within statistical noise. This is a direct consequence of the meta-learner architecture: the ensemble’s December output converges toward its LR meta-learner’s prediction under the higher-delay distribution (25.5%), where LR’s calibrated linear boundary generalizes marginally better than the weighted combination of diverse base models. The ensemble’s superiority is confirmed on the primary November evaluation (AUC 0.9295 vs. 0.9290) and by McNemar’s test ($p = 0.768$ — structurally coupled, not independently inferior).

The stability of **TAIL_DELAY_RATE** as the third most important feature across both temporal holdouts validates that individual aircraft operational history encodes genuine and persistent pre-flight delay signal, independent of seasonal variation or holiday-driven distribution shift. Its rank below **DEP_DELAY** is mechanistically expected: **DEP_DELAY** reflects the immediate flight state at departure, whereas **TAIL_DELAY_RATE** captures chronic reliability patterns over the training period — the two features are complementary proxies for different time horizons of delay causation.

The improvement in regression R^2 from November (0.878) to December (0.887), despite higher absolute delay rates, reflects the predictability structure of holiday-season disruptions: delays during Thanksgiving and Christmas periods cluster around known peak windows, reducing residual variance and making delay magnitude more estimable than in the lower-variance November baseline.

One limitation meriting further investigation is the 3.5% cold-start rate for **TAIL_DELAY_RATE** in December, where

unknown tail numbers fall back to the global mean (0.2255). Future work should evaluate whether a carrier-level fallback — replacing the global mean with the mean delay rate for the operating carrier — reduces prediction error on newly introduced aircraft.

VII. CONCLUSION

This paper presented a dual-task machine learning framework for pre-departure flight delay prediction combining binary classification and regression under strict temporal validation. Using 300,000 stratified records from the BTS dataset, the stacking ensemble achieved AUC 0.9295 on the November holdout and 0.9254 on the December holiday stress test — outperforming prior works reporting 0.87–0.91 under more lenient random-split evaluations. LightGBM achieved $R^2=0.8874$ and RMSE=15.30 min on the December stress test.

The novel `TAIL_DELAY_RATE` feature, capturing individual aircraft delay history at the tail number level, was validated by SHAP as the third most important feature on temporally unseen data — demonstrating that aircraft operational history is a genuine and generalizable pre-flight signal absent from prior literature. The temporal dual holdout protocol revealed that random splits inflate AUC by 0.016–0.032 across tree-based and deep learning models, underscoring the importance of honest evaluation in aviation ML research.

Several limitations of this work should be acknowledged. The framework is evaluated exclusively on U.S. domestic flights across four months, and its generalizability to international routes or other time periods remains to be validated. The absence of real-time weather forecast data as a pre-departure input limits predictive power for weather-driven disruptions. Additionally, `TAIL_DELAY_RATE` cannot be computed for newly introduced aircraft with no training history, relying instead on a global mean fallback.

Future work will explore real-time feature updates, integration of weather forecasts as pre-departure inputs, extension to international aviation datasets, and deployment in a production inference pipeline with live BTS data streams.

REFERENCES

- [1] Federal Aviation Administration, “Cost of flight delays,” FAA, Washington, DC, USA, Tech. Rep., 2019.
- [2] Eurocontrol, “European airline delay cost reference values,” Eurocontrol Performance Review Unit, Brussels, Belgium, Tech. Rep., 2022.
- [3] B. Campanelli, P. Fleurquin, A. Arranz, and I. Etxebarria, “Comparing the modeling of delay propagation in the us and european air traffic networks,” *Journal of Air Transport Management*, vol. 56, pp. 12–22, 2016.
- [4] V. Deshpande and M. Arikan, “The impact of airline flight schedules on flight delays,” *Manufacturing and Service Operations Management*, vol. 14, no. 3, pp. 423–440, 2012.
- [5] S. A. A. AlBassam and D. N. AlShahrani, “Flight delay prediction: Evaluating machine learning algorithms for enhanced accuracy,” *PLoS One*, vol. 20, no. 1, 2025.
- [6] I. Hatipoğlu and O. Tosun, “Predictive modeling of flight delays at an airport using machine learning methods,” *Applied Sciences*, vol. 14, no. 3, 2024.
- [7] M. Zoutendijk and M. Mitici, “Probabilistic flight delay predictions using machine learning and applications to the flight-to-gate assignment problem,” *Aerospace*, vol. 8, no. 5, 2021, art. no. 131.

- [8] D. B. Bisandu, “Novel deep learning approaches for flight delays prediction,” Ph.D. dissertation, Cranfield University, Cranfield, UK, 2023.
- [9] M. Mamdouh, M. Ezzat, and H. A. Hefny, “A novel intelligent approach for flight delay prediction,” *Journal of Big Data*, vol. 10, no. 1, 2023.
- [10] J. Singh and M. D. Jayaprakash, “Flight delay prediction for indian air carriers with explainable artificial intelligence,” in *Proc. 2022 Int. Conf. Applied Artificial Intelligence and Computing (ICAAIC)*. IEEE, 2022.
- [11] P. Hu, J. Zhang, and N. Li, “Research on flight delay prediction based on random forest,” in *Proc. IEEE 3rd Int. Conf. Intelligent Transportation Engineering (ICITE)*, 2021.
- [12] A. Sheikholeslami, H. Khaksar, and S. Z. Siabil, “Predicting flight delays by using a meta-classifier,” in *Proc. Transportation Research Board Annual Meeting*, 2016.
- [13] Bureau of Transportation Statistics, “Airline on-time performance data,” <https://www.transtats.bts.gov>, 2024, accessed: May 2026.
- [14] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.
- [15] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T. Liu, “Lightgbm: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems*, 2017, pp. 3146–3154.
- [16] T. Chen and C. Guestrin, “Xgboost: A scalable tree boosting system,” in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [17] S. O. Arik and T. Pfister, “Tabnet: Attentive interpretable tabular learning,” in *Proc. AAAI Conf. Artificial Intelligence*, vol. 35, 2021, pp. 6679–6687.
- [18] D. H. Wolpert, “Stacked generalization,” *Neural Networks*, vol. 5, no. 2, pp. 241–259, 1992.
- [19] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, “Smote: Synthetic minority over-sampling technique,” *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.
- [20] S. M. Lundberg and S. Lee, “A unified approach to interpreting model predictions,” in *Advances in Neural Information Processing Systems*, 2017, pp. 4765–4774.
- [21] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.
- [22] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, 2nd ed. New York, NY: Springer, 2009.
- [23] D. Micci-Barreca, “A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems,” *ACM SIGKDD Explorations Newsletter*, vol. 3, no. 1, pp. 27–32, 2001.
- [24] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, “Scikit-learn: Machine learning in python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [25] H. He, Y. Bai, E. A. Garcia, and S. Li, “Adasyn: Adaptive synthetic sampling approach for imbalanced learning,” in *Proc. IEEE Int. Joint Conf. Neural Networks (IJCNN)*, 2008, pp. 1322–1328.